Glossary: Cross-Domain Circuit Analysis

Purpose: Plain-language definitions of key concepts used in the paper, for explaining the work to others.

Companion to: `CROSS_DOMAIN_CIRCUIT_PAPER.md`

Table of Contents

Jaccard Similarity

Quick definition: A number between 0 and 1 that measures how much two sets overlap.

Formula: $$J(A, B) = \frac{|A \cap B|}{|A \cup B|} = \frac{\text{size of overlap}}{\text{size of combined set}}$$

How to read the values:

- **J = 0** → the two sets share nothing
- **J = 1** → the two sets are identical
- **J = 0.5** → half the elements are shared

Worked example:

Set A = {apple, orange, banana, grape} Set B = {apple, orange, pear, kiwi}

- Intersection: {apple, orange} → size 2
- Union: {apple, orange, banana, grape, pear, kiwi} → size 6
- **J = 2/6 ≈ 0.33**

How we use it: Every circuit has a set of SAE features that activate during the prompt. We compute Jaccard over those feature sets to ask "how similar are these two circuits?"

Example results from the paper:

Comparison	Jaccard	Interpretation
Gold vs Sodium (same template, different facts)	0.60	Circuits share 60% of features
Gold template-A vs Gold template-B (same fact, different templates)	0.30	Only 30% shared
Chemistry vs Geography (different domain)	0.11	Very different circuits
Gemma Geography prompts (within-domain)	0.40	Highest within-domain convergence
Gemma History prompts (within-domain)	0.11	Lowest within-domain convergence

Why we chose it:

- Simple and interpretable
- Doesn't care about set size (two small sets with perfect overlap still score 1.0)
- Ignores element ordering and magnitude

Limitation: Treats all features as equally important. A strongly-activated feature counts the same as one that barely fires.

Activation Magnitude

Quick definition: The summed absolute activations of a circuit's SAE features, computed per layer or for the whole circuit. It quantifies how strongly the circuit's features fire at each depth.

How we compute it:

- **Per-layer activation magnitude** = sum of absolute activations across all of the circuit's SAE features at that layer
- **Total activation magnitude** = sum of per-layer magnitudes across the entire circuit
- **Per-layer activation share** = (magnitude at layer L) / (circuit total), i.e. what fraction of the circuit's activation sits at layer L

How it looks in the paper:

Layer activation profiles plot per-layer activation share against layer depth for each circuit. We found two distinct patterns:

- **Gemma is front-loaded:** 54% of total activation magnitude is in layers 0-12 (first half). The cumulative share reaches 50% by L11.
- **Qwen is back-loaded:** only 31% of the total is in the first half. The cumulative share does not reach 50% until L28.

These are different depth strategies, but both produce comparable factual recall.

Key property: Layer activation profiles are **architecture-determined, not domain-determined.**

- Within-model cosine similarity (across chemistry/geography/history): **0.978**

- Between-model cosine similarity: **0.696**
- The architecture gap is roughly 14× the domain gap

Why this matters: Layer activation profiles are robust to prompt-template confounds (see Section 5.9). Feature IDs change when you rephrase prompts, but *where* computation happens across layers stays architecturally locked. That's why the architecture-dominance claim survives the format-variation critique.

Bottleneck Penalty

Quick definition: The empirical finding that **higher activation magnitude at a bottleneck layer correlates with* worse *model confidence.** The name is a memorable label; the theoretical grounding is information bottleneck theory (Tishby & Zaslavsky, 2015), which predicts that over-compression at an intermediate layer limits the information downstream layers can use for prediction.

Important note on framing: This is a **correlation with a theoretical interpretation, not a proven causal mechanism**. Activation magnitude is not a conserved quantity (see "A Note on Terminology"). The finding is grounded in information bottleneck theory, not in any resource metaphor.

The core correlations (Gemma-2-2B):

Layer	Role	Correlation with confidence	Direction
L6	Primary bottleneck	$r = -0.684$, $p < 0.0001$	Negative (penalty)
L10	Mid-layer	$r = -0.601$	Negative
L1	Input	$r = +0.634$	Positive
L13	Post-bottleneck	$r = +0.601$	Positive
L16	Post-bottleneck	$r = +0.598$	Positive

More activation at L6 → worse predictions. More activation at L13/L16 → better predictions.

The intuition (doorway analogy, use cautiously): Think of a narrow doorway in a hallway.

- Information has to squeeze through the doorway (bottleneck at L6)
- The *useful assembly* happens in the rooms after the doorway (L13, L16)
- When too much is compressed at the doorway, less useful information reaches the downstream rooms for assembly

This is a metaphor to build intuition. The **formal version** comes from information bottleneck theory: the mutual information $I(X; T_L)$ between input X and the representation at layer L bounds what downstream layers can recover. Higher compression at the bottleneck layer reduces this mutual information, which limits downstream evidence accumulation.

Note: The bottleneck both filters information (potentially helpful, by discarding noise) AND limits downstream information (potentially harmful, if it discards signal). The paper's finding is that on average the harm dominates: circuits with heavier bottleneck activation tend to produce less confident predictions.

What we ruled out:

- **Bottleneck convergence** (how sharply paths funnel): $r = 0.27$, NOT significant
- **Bottleneck activation share** (what fraction of the circuit total sits at the bottleneck): $r = 0.26$, NOT significant
- **Total activation magnitude**: $r = 0.53$, Bonferroni-significant

So it is not about having a "sharper" or "cleaner" bottleneck; what matters is how much activation is concentrated there.

Why this matters: This is one of the paper's most novel contributions because it **connects circuit structure directly to behavior**. Earlier work (Meng et al. 2022) showed specific layers matter for factual recall via causal intervention. Our bottleneck penalty is the correlational version: just by measuring *where* activation concentrates in a circuit, you can predict *how confident* the model will be.

Qwen3-4B contrast: Qwen shows zero Bonferroni-significant layer-activation-confidence correlations. Its late-bottleneck architecture (L22-25) diffuses the confidence signal across many layers instead of localizing it. The bottleneck penalty appears specific to early-bottleneck architectures like Gemma.

One-line summary: Bottleneck penalty = an empirical correlation (higher L6 activation share $\rightarrow$ lower confidence) grounded in information bottleneck theory (Tishby 2015), not a resource mechanism.

How to explain it to someone: > "We found that the more a circuit's activation is concentrated at its bottleneck layer (L6 in Gemma), the less confident its prediction tends to be: r = -0.684, Bonferroni-significant. This is consistent with information bottleneck theory, which predicts that heavy compression at intermediate layers limits the information downstream layers can use. We call the pattern the 'bottleneck penalty' for memorability, but it's a correlational finding with a theoretical interpretation, not a proven causal mechanism."

Architecture Dominance

Quick definition: The empirical observation that within-model circuits cluster about 14× more tightly in layer-activation-profile similarity than between-model circuits, with knowledge domain modulating the profile by less than 2%. In short: which model you are using matters far more for circuit structure than what you are asking the model about.

The core numbers (Section 5.5.1 and 6.1):

Comparison	Cosine similarity on layer activation profiles	Interpretation
Within Gemma, across chemistry/geography/history	0.978	Nearly identical shape
Within Qwen, across chemistry/geography/history	0.978	Nearly identical shape
Between Gemma and Qwen (depth-normalized)	0.696	Substantial structural gap
Gap	0.282	~14× the within-model domain spread

The within-model figure clusters across all three knowledge domains. The between-model figure persists even after normalizing layer indices to a common 0-1 depth scale, so it is not an artifact of Gemma having 26 layers and Qwen having 36. A Mann-Whitney U test on the two distributions of pairwise similarities returns $p < 0.000001$, with non-overlapping 95% bootstrap confidence intervals.

Beyond layer activation profiles: 13 of 16 structural metrics (node counts, edge density, layer breadth, path depth, bottleneck depth, and others) differ significantly between Gemma and Qwen circuits under a Bonferroni-corrected Mann-Whitney test. Peak activation layer shows perfect rank separation: every Gemma circuit peaks earlier than every Qwen circuit, with no overlap across the 60-circuit dataset.

Why this is a load-bearing claim: The natural prior is that factual recall circuits differ by what they recall (chemistry features for chemistry prompts, geography features for geography prompts). Architecture dominance flips that prior. The same model produces structurally near-identical circuits across all three domains, while a different model on the same prompts produces a different structural signature. Domain is a small perturbation on top of an architecture-determined backbone.

Robustness to format confounds: Because feature IDs change when prompt templates are rephrased (Section 5.9), feature-overlap metrics can be inflated by template repetition. Layer activation profiles avoid this confound: they describe where computation happens across layers, not which specific features fire. Architecture dominance survives the format-variation critique that weaker overlap-based claims do not.

How to explain it to someone: > "We compared the layer-by-layer activation distribution of 60 circuits, 30 from each model, spanning three knowledge domains. Within a single model, circuits cluster at 0.978 cosine similarity regardless of domain. Between models, that drops to 0.696, even after we normalize layer depth. The gap is about 14 times larger than the spread we see within either model when we change the topic. So architecture, not knowledge, dictates the structural shape of factual recall circuits in these models."

Universal Bottleneck Features

Quick definition: SAE features that act as bottlenecks (path convergence at or above 60% in the traceback analysis) across all three knowledge domains within a single model architecture. They are not knowledge stores but shared routing infrastructure that domain-specific circuits pass through.

The core inventory (Section 5.3):

- Gemma-2-2B: 6 universal bottleneck features
- Qwen3-4B: 15 universal bottleneck features
- Cross-model overlap: 0 features (the two sets are disjoint)

Gemma's six:

Feature ID	Semantic role
L0_F1813559	Code and file-keyword tokens
L0_F74438300	Early-layer routing (mixed)
L3_F5150441	HTML formatting tags
L4_F110446948	Place names
L6_F2586668	Code snippets
L24_F88478228	Lithuanian place names

Qwen's 15 span layers L15 to L34, sitting squarely in the late-bottleneck region characteristic of Qwen's architecture. By semantic role, the dominant categories are CODE (about 20%) and LANGUAGE (about 20%), with explicitly domain-specific features (chemistry, geography, history) accounting for only about 13% of universal bottlenecks combined.

Why this matters (Section 6.2): Universal bottlenecks are surprising on two counts. First, the dominant semantic categories (code, language formatting, place names) are not the categories you would predict from the prompts themselves. Many circuits route encyclopedic factual recall through features that originally encode code structure or HTML markup. Second, the lack of cross-model overlap means there is no shared universal feature set across models, despite both models exhibiting the same architectural pattern of having universal bottlenecks at all. Each architecture builds its own routing layer from whatever features its training landed on.

The high-leverage intervention hypothesis: If a handful of universal bottlenecks carry many circuits, then steering or ablating those features should produce out-sized causal effects relative to randomly-chosen features. The steering experiments (D4-D6, see Three-Tier Dissociation) test this hypothesis and find it only partially supported: structural universality predicts where compression happens, but not how much downstream behavior changes.

How to explain it to someone: > "Within each model, a small set of SAE features sits on the path of nearly every factual-recall circuit we examined, regardless of whether the prompt is about chemistry, geography, or history. Gemma has six such features; Qwen has fifteen. The two sets do not overlap, and neither is dominated by domain-specific concepts. Most of them encode code structure, language formatting, or place names. They look more like routing infrastructure than knowledge stores."

Traceback Graphing

Quick definition: The paper's core analytical method. A backward breadth-first search from a circuit's top output tokens through SAE feature activations and attribution edges, scored with geometric decay, that identifies the critical paths a model uses to assemble its prediction and the bottleneck features those paths converge on.

The algorithm (Section 3.2):

1. Identify the top-K output tokens by final-layer logits (default K=5). These become the anchor nodes for the search.
2. Run a backward breadth-first search from each anchor through the attribution graph, layer by layer.
3. Score each predecessor node by `activation × edge_weight × downstream_score^0.8`. The exponent 0.8 is a geometric-decay factor applied to the recursively-accumulated downstream score.
4. Record the full path from each output anchor back through every visited intermediate node.
5. Flag a feature as a "bottleneck feature" for the circuit if it appears on at least 60% of the recorded paths.

Why geometric decay: Without it, multiplying raw activation scores across 20-plus layers produces exponential overflow at deeper search depths. The 0.8 exponent preserves the relative ranking of paths while keeping accumulated scores in a numerically tractable range. The decay is applied uniformly, so it does not bias the analysis toward any particular layer.

What the method produces:

- **Critical paths:** Ranked sequences of features that contributed most strongly to each top-K output token.
- **Bottleneck features:** The features that show up on most of those paths, identified by the 60% convergence threshold.
- **Convergence statistic:** The fraction of paths that share each candidate feature, which becomes the unit of analysis for cross-domain comparison (Section 5.3) and for the universal-bottleneck inventory.

A useful diagnostic property: Top-K and bottom-K tracing (anchoring on the highest- and lowest-logit output tokens respectively) tend to converge on the same bottleneck features. This is consistent with the idea that circuits share routing infrastructure across all output tokens at the top of the logit distribution, rather than building a separate pathway per candidate token.

Where the method appears in the paper: Traceback graphing is the analytical engine behind every structural claim in the paper. Bottleneck inventories (Section 5.3), within-model convergence (Section 5.4), and the steering selection criteria for D4 and D5 (Section 5.8) all derive from features that traceback graphing flagged as bottlenecks.

How to explain it to someone: > "We start at the model's predicted output tokens and walk backward through the attribution graph, layer by layer, scoring each step with activation magnitude times edge weight and applying a geometric decay so the scores stay numerically stable across 20-plus layers. The output is a set of critical paths from input to output. Any feature that sits on more than 60% of those paths gets called a bottleneck. The same machinery identifies bottleneck features, lets us compare circuits across domains and models, and supplies the candidate features for our steering experiments."

Three-Tier Dissociation

Quick definition: A pattern that emerged from 80 single-feature steering experiments: the structural property that best predicts distributional impact (essential-pathway membership) does not predict whether output text actually changes, and text-level changes are governed instead by domain-specific output entropy. Three predictors, three different effects, no single criterion that wins on all of them.

The three tiers (Section 5.8.3):

Tier	Predictor	What it predicts	What it does not predict
1	Essential-pathway membership (topology)	Mean KL divergence 1.448 (the largest distributional perturbation of any selection criterion)	Text-change rate (26.7%, comparable to frequency-selected features)
2	Cross-circuit frequency	A feature appearing in many circuits	Causal influence. L2_F25751073 (top frequency, 11 circuits) produced 0 text changes
3	Output entropy by domain	Text-level susceptibility: chemistry 0%, geography 20%, history 60%	Anything about which features were steered

What the dissociation rules out: It is tempting to assume that the features that show up most often across circuits, or the features that sit on the critical pathway, are also the ones that most reshape model behavior when perturbed. Neither relationship holds cleanly. The most-frequent feature (L2_F25751073) was causally inert at strength ±20. The strongest single perturbation in the dataset, L25_F50014975 on a geography prompt, produced KL = 12.72 and flipped the output to "Tokyo", but it was selected by essential-pathway membership, not by frequency, and an equivalent perturbation on a chemistry prompt produced no text change at all.

Why redundancy absorbs perturbations: Roughly 94.1% of nodes per circuit sit off the essential pathway. They form redundant scaffolding that can compensate for an isolated perturbation by re-routing activation through alternate edges. Essential-pathway features create the largest local effects (high KL), but the downstream layers smooth those effects back toward the original distribution before they reach the output. Whether a perturbation breaks through to the output is then governed by how concentrated the output token distribution was to begin with, a property of the prompt's domain.

The asymmetry by domain: History prompts are most susceptible (60% text-change rate) because their output distributions are flatter, with multiple plausible year tokens competing near the top. Geography prompts are intermediate (20%) because a single canonical answer usually dominates but can be displaced. Chemistry prompts show 0% text change because the output entropy is so low that no single-feature perturbation at ±20 strength reshuffles the argmax.

Why this is a finding rather than a failed validation: The natural expectation going in was that essential-pathway features would dominate on every metric: largest KL, highest text-change rate, most consistent across domains.

The first prediction holds, the second does not, and the third turns out to be controlled by the prompt rather than the feature. That decomposition into three independent axes is the dissociation. It implies that any single metric for "important feature" is incomplete, and that causal intervention studies need to report all three.

How to explain it to someone: > "We ran 80 steering experiments with three different selection criteria. Essential-pathway features produced the strongest distributional perturbations on average, with mean KL of 1.448. But they did not flip output text any more reliably than features selected by cross-circuit frequency, both landing around a quarter of experiments. And the rate of text changes turned out to depend almost entirely on the prompt domain: chemistry 0%, geography 20%, history 60%, regardless of which features we steered. Three tiers of causal effect, three different predictors, no single metric that wins on all three."

Cosine Similarity on Layer Activation Profiles

Quick definition: A number between 0 and 1 measuring whether two circuits have the same *shape* of layer-by-layer activation distribution, regardless of total magnitude.

What the inputs are: Each circuit produces a vector of N numbers (26 for Gemma, 36 for Qwen), the proportion of the circuit's total activation magnitude at each layer. Example:

```
Gemma circuit → [0.08, 0.12, 0.15, 0.09, 0.06, 0.04, 0.03, 0.02, ...]
```

These sum to 1.0 (they're fractions).

Why cosine specifically:

1. **Shape over magnitude.** Measures the *angle* between vectors, not their length. Two circuits concentrating activation at L6 will score as similar even if one has much larger absolute magnitudes. We care about *where* computation happens, not how much.
2. **Standard in ML.** Used ubiquitously for comparing embeddings, feature vectors, and activation patterns.
3. **Interpretable.** 1 = identical shape, 0 = completely different shape.

Alternatives we didn't use and why:

- **Euclidean distance:** dominated by absolute magnitude differences, not shape
- **KL divergence:** asymmetric, treats vectors as probability distributions (which is OK since ours sum to 1, but harder to interpret as a "similarity")
- **Correlation:** adds a mean-centering step that's harder to interpret here

Precedents in the literature:

- **Representational Similarity Analysis (RSA):** Kriegeskorte, Mur, & Bandettini (2008), *Frontiers in Systems Neuroscience*. Standard for comparing activation patterns across brains/models/conditions.
- **Centered Kernel Alignment (CKA):** Kornblith, Norouzi, Lee, & Hinton (2019), *ICML*, "Similarity of Neural Network Representations Revisited." Modern method for comparing activations across networks.
- **Cosine on SAE features:** Bricken et al. (2023), Templeton et al. (2024) use cosine similarity on feature activations.

What's novel in our application: Applying cosine similarity specifically to **per-layer activation shares aggregated across a circuit**, rather than comparing individual activation vectors. It's a reasonable extension of standard practice.

The key results (Section 5.5.1):

- Within-model cosine similarity: **0.978**

- Between-model cosine similarity: **0.696** (after depth normalization)
- Mann-Whitney U test: **p < 0.000001** (distributions don't overlap)
- Bootstrap 95% CI on within-model mean: **[0.975, 0.981]**

Where we're vulnerable to reviewer critique:

- We didn't formally benchmark cosine against KL divergence or Earth Mover's Distance. A reviewer could reasonably ask "would the pattern hold with a different metric?"
- The "~14×" claim divides the between-model gap (0.282) by the within-model domain gap (~0.02). This is a ratio of mean cosine gaps, not a formal variance decomposition, and the paper words it accordingly.

How to explain it: > "Cosine similarity measures whether two circuits concentrate their computation at the same layers: we care about the shape of where work happens, not the absolute amount. This extends representational similarity analysis (Kriegeskorte 2008) and modern activation-comparison methods like CKA (Kornblith 2019). Our finding that within-model scores cluster at 0.978 while between-model scores drop to 0.696 is statistically robust (Mann-Whitney p < 0.000001)."

A Note on Terminology: Why "Activation Magnitude," Not "Energy"

Quick definition: Earlier drafts of this work used "energy" as shorthand for summed feature activation magnitudes. The published version uses "activation magnitude" throughout. This entry records why.

The problem with "energy":

- Neural networks have no conservation law for activations. Layer 6 can activate many features without "using up" anything layer 13 needs. A reader who takes "energy" literally infers a budget that does not exist.
- "Activation energy" is an established chemistry term (the Arrhenius barrier a reaction must overcome). Chemistry is one of this paper's three knowledge domains, so the collision is direct.
- Energy-based models (LeCun, Hinton) use "energy" as a scalar objective over configurations, a different concept again. Borrowing the word invites confusion with that literature.

What is actually computed: for a circuit and a layer, the summed absolute activation of the circuit's SAE features at that layer. Summing over layers gives the circuit total; dividing a layer's sum by the total gives that layer's activation share. Nothing in the analysis depends on any physical analogy.

What survives the renaming unchanged: every number. The within-model profile similarity (0.978), the between-model similarity (0.696), the L6 correlation with confidence (r = -0.684), and the front-loaded/back-loaded contrast (54% vs 31% in the first half of layers) are all statements about activation magnitudes and are unaffected.

The theoretical grounding for the bottleneck penalty comes from information bottleneck theory (Tishby & Zaslavsky 2015; Shwartz-Ziv & Tishby 2017), which is stated in terms of mutual information, a quantity with a precise meaning here, unlike "energy."

Steering Experiments: How Features Were Selected

Quick definition: Steering is a causal validation technique where we clamp a single SAE feature's activation to a fixed value (positive = amplify, negative = suppress) and measure whether the model's output changes. We ran 80 such experiments across three batches, each using a different criterion to choose which features to steer.

The basic mechanism: For each experiment, we made one Neuronpedia API call with one feature, one prompt, and one strength value:

```
POST /api/steer
```

```
prompt   = "The Titanic sank in"
feature  = L7_F4828270  (one feature at a time, never combined)
strength = +20           (or -20; ±20 was used in every experiment)
```

The API returned both the unsteered (`DEFAULT`) and steered (`STEERED`) outputs along with token-level logprobs. We measured: (a) text change (binary), (b) KL divergence between baseline and steered token distributions, (c) logprob shift in the top token.

Three batches, three different selection criteria:

Batch	Criterion	Source pool	Features	Experiments
D4	Cross-circuit frequency, ranks 1-5	Stage 1.5 bottleneck library (244 features that appeared as bottlenecks across the 60-circuit dataset)	L0_F1813559, L3_F5150441, L4_F110446948, L6_F2586668, L24_F88478228	20
D5	Cross-circuit frequency, ranks 6-10 (extending D4)	Same library	L1_F99962728, L2_F25751073, L5_F7993995, L7_F4828270, L9_F125286525	30
D6	Essential-pathway membership (topology)	1,000 features extracted from minimal-pathway analysis on 30 Gemma circuits	L0_F64712375, L1_F1736314, L21_F5479683, L24_F18002975, L25_F50014975	30

Cross-circuit frequency = the number of distinct circuits (out of 60) in which a feature appears as a bottleneck (convergence ≥ 60% in the traceback analysis). High frequency = "this feature shows up everywhere."

Essential-pathway membership = the number of distinct circuits whose minimum-viable input-to-output pathway includes this feature. High pathway membership = "this feature is on the critical route, not the redundant scaffolding." Only ~6% of nodes per circuit are on the essential pathway.

Why three batches:

D4 was the initial validation. D5 expanded it because only 50% of D4 experiments produced text changes and we wanted more statistical power. D6 was added when an unexpected finding emerged from D5: the most-frequent feature (L2_F25751073, 11 circuits) produced **zero** text changes, while a less-frequent one (L7_F4828270, 9 circuits) was the most causally effective. This dissociation between frequency and causation motivated testing a different criterion (topology) to see whether essential-pathway features would do better.

Pre-experiment cross-check: Of the 10 frequency-selected features in D4+D5, only 5 were on essential pathways (L0_F1813559, L1_F99962728, L2_F25751073, L3_F5150441, L24_F88478228). The other 5 were frequent but NOT essential. They appear often but aren't on critical paths. This split made D6 a clean comparison.

What was NOT varied:

- **Strength.** All 80 experiments used ±20 only. No dose-response curve. (See "Why ±20?" below.)
- **Number of features per intervention.** Always one feature at a time. No compound steering.
- **Random seed.** All experiments used `seed=42, temperature=0` for reproducibility.

Why ±20 strength specifically?

The Neuronpedia steering API accepts strengths from approximately -50 to +50. We used ±20 across all 80 experiments. Three reasons:

1. **It avoids two empirical failure modes.** At ±5 or ±10, most features produce no visible text change: too weak to distinguish causally-inert features from under-stimulated ones. At ±50 or higher, the model often produces nonsensical output ("activation overflow"), with effects that are real but uninterpretable. ±20 is in the middle: strong enough to detect causal influence, weak enough that surviving outputs remain grammatical.

2. **It is a community-convention default.** Templeton et al. (2024) used clamping values roughly 5-10× a feature's natural maximum for the "Golden Gate Bridge" demonstration. Turner et al. (2023, activation engineering) used coefficients typically in the 1-15× range. Neuronpedia's documentation suggests ±10 to ±30 as illustrative defaults. ±20 sits in the middle of this established range. Reporting ±20 also keeps our results directly comparable to other groups using the same convention.

3. **Practical: API rate-limit budget.** Each steering call costs ~36 seconds at Neuronpedia's 100-call/hour limit. Our total budget was 80 calls. We spent it on breadth (15 features × 3-6 prompts) rather than depth (a full dose-response curve at ±5/±10/±20/±50). Building the dose-response curve is listed as Future Work.

Empirical validation that ±20 was a reasonable choice:

Batch	Text change rate at ±20	Interpretation
D4	50% (10/20)	Some features change outputs, others don't; clean separation
D5	23% (7/30)	Most features ineffective at this strength; selection matters
D6	27% (8/30)	Comparable to D5; different selection, similar overall rate

These rates fall in the useful 20-50% band: not so low that everything looks inert, not so high that everything looks effective. Both ends of the spectrum would have hidden the three-tier dissociation finding by collapsing it into "all features causal" or "no features causal."

Limitations of single-strength testing:

We do not know whether the three-tier dissociation persists at other strengths. It is possible that at ±50, essential-pathway features dominate text-change rates as well as KL divergence (because circuit redundancy can no longer absorb the larger perturbation). It is also possible that at ±5, only frequency-on-pathway features show any effect. The single-point sample at ±20 captures one slice of an unmeasured curve.

The headline finding (the "three-tier dissociation"):

Group	Selection method	Text change rate	Mean KL divergence
Essential-pathway features (D6)	Topology	26.7% (8/30)	1.448
D5 features that happen to be ON pathway	Frequency-on-pathway	18.8% (3/16)	~0.4
D5 features that are OFF pathway	Frequency-off-pathway	33.3% (6/18)	~0.4

Essential-pathway features produced the strongest distributional perturbations (highest KL) but circuit redundancy absorbed most of the perturbation before the output layer. Output determinism by domain (chemistry 0%,

geography 20%, history 60%) governed text-level susceptibility regardless of which features were steered.

One-line summary: We steered 15 unique features (5 per batch × 3 batches) at ±20 strength on 3-6 prompts each, with each batch chosen by a different selection criterion (frequency × 2, then topology). The three-tier dissociation finding emerged from comparing the three batches against each other.

How to explain it to someone: > "We did 80 single-feature steering experiments split into three batches. The first two batches selected features by how often they appeared across our 60-circuit dataset; the third selected by whether they sit on the essential information pathway. Each feature was steered at ±20 strength on three target prompts (one per knowledge domain). The point of using three different selection criteria was to test whether structural importance metrics, frequency vs topology, actually predict causal influence. Neither does very well, which is itself a finding."

Direct Logit Attribution: How the Output Token is Chosen

Quick definition: The mechanism by which activations of output-layer SAE features get translated into a chosen vocabulary token. Each feature has a "decoder direction" in residual-stream space; that direction projected through the unembedding matrix tells you which tokens the feature pushes toward and which it pushes away from. The model selects whichever token has the highest resulting logit.

The full path from output features to chosen token:

```
Output-layer SAE features (L25 in Gemma, L35 in Qwen)
            ↓
   Each feature has a decoder direction d_f (a vector in residual-stream space)
            ↓
   Sum of (activation × d_f) reconstructs the residual stream
            ↓
   Final LayerNorm
            ↓
   Multiply by unembedding matrix W_U (vocab_size × hidden_dim)
            ↓
   One logit per vocabulary token
            ↓
   argmax (or softmax sample at temperature > 0) → the chosen token
```

The math behind individual feature contributions:

For a single feature `f` with decoder direction `d_f`, activation magnitude `a_f`, and unembedding column `W_U[:, t]` for token `t`:

```
logit_contribution(f, t) = a_f × (d_f · W_U[:, t])
```

This dot product is essentially the alignment between the feature's decoder direction and the token's unembedding row. **High positive value** = "this feature pushes toward this token." **High negative value** = "this feature suppresses this token." **Near zero** = "this feature has no direct effect on this token."

This calculation is called **direct logit attribution (DLA)**, sometimes also called "logit lens applied to SAE features."

Two kinds of "output node" in our pipeline:

1. **Logit nodes:** these represent specific vocabulary tokens. The Top-K (K=5) highest-logit tokens become the anchor nodes that traceback graphing starts from. For "The capital of Japan is", the top logit nodes might be `" Tokyo"`, `" the"`, `" a"`, `" called"`, `" known"`.

2. **Final-layer SAE feature nodes:** SAE features at the last layer (L25 for Gemma, L35 for Qwen) whose decoders project toward those top-K tokens. Their connections to logit nodes in the attribution graph are weighted by direct logit attribution.

Why output-feature convergence is meaningful (the §5.4 finding):

Same-domain circuits share **67% of their output features** in Qwen history (vs only 15% of path features). The mechanism: different history prompts ("WWII ended in", "Columbus reached the Americas in") need to produce different tokens (1945, 1492), but those tokens share decoder structure (both 4-digit years, both common in encyclopedic text). The same output features push toward the year-token region of vocabulary space. What differs across prompts is how earlier layers route information to those shared output features.

This is the formal basis for "convergent outputs, divergent paths."

The selection step (at temperature=0):

`argmax(logits)`. That's it. No beam search, no sampling. The token with the highest logit wins. This is why steering can flip predictions: shifting one feature's activation ripples through the decoder math, changes some logits, and can flip which token wins the argmax.

Why the bottleneck penalty connects to this:

If activation concentrates at L6 (compression bottleneck), less informative residual-stream content reaches L25 where the unembedding actually consumes it. The output features at L25 still fire (but on degraded input), producing a flatter logit distribution (lower confidence). The information bottleneck framing captures this: I(X; T_L25) is bounded by I(X; T_L6), and a flatter logit distribution means less peaked argmax probability.

Tools that compute DLA for you:

- **Neuronpedia feature dashboards** show top positive and negative logits per feature in their `top_logits` and `bottom_logits` fields
- **TransformerLens** has `apply_ln_to_stack()` and unembedding helpers for projecting activations through the final layers manually
- The Neuronpedia feature explanation API returns these fields when available

One-line summary: Each output-layer feature has a decoder direction that, when projected through the model's unembedding matrix, produces a contribution to every vocabulary token's logit. The chosen token is just argmax of the sum of these contributions across all active features.

How to explain it to someone: > "The model doesn't 'pick' tokens directly from features. Each output-layer feature has a learned vector that encodes which tokens it pushes toward when active. Those vectors get projected through the unembedding matrix to produce logits, and the highest-logit token wins. Two features that look completely different in semantic terms can both push toward 'Tokyo' if their decoder vectors happen to align with the unembedding row for 'Tokyo'. That is why we see same-domain prompts converging on shared output features even when their internal processing diverges."

How to Extend This Document

When a new question comes up:

1. Add entry to the Table of Contents
2. Follow the structure: Quick definition → Details → Example → Why it matters
3. Keep plain-language definitions at the top of each section so readers who only want the gist don't have to read the full explanation
4. Cross-reference with specific sections of `CROSS_DOMAIN_CIRCUIT_PAPER.md` where relevant